

Trường ĐH Công Thương TP. HCM Khoa Công nghệ Thông tin Bộ môn Kỹ thuật phần mềm Công nghệ .NET	BÀI 5. MÔ HÌNH MVVM	
--	--------------------------------	--

A. MỤC TIÊU

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Trình bày được cấu trúc và vai trò của các thành phần trong mô hình MVVM.
- Áp dụng được cơ chế Data Binding và INotifyPropertyChanged để đồng bộ dữ liệu giữa View và ViewModel.
- Sử dụng thành thạo ObservableCollection<T> để quản lý và hiển thị danh sách dữ liệu động.
- Vận dụng được ICollectionView để thực hiện lọc và sắp xếp dữ liệu trên giao diện mà không làm thay đổi dữ liệu gốc.
- Xây dựng được ứng dụng WPF đơn giản theo mô hình MVVM, đảm bảo tách biệt rõ View – ViewModel – Model.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SINH VIÊN

STT	Thiết bị - Phần mềm – Thư viện	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	
2	Microsoft Visual Studio 2013 trở lên	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Mô hình MVVM

Định nghĩa: Mô hình MVVM (Model – View – ViewModel) là mô hình kiến trúc thiết kế ứng dụng WPF giúp:

- Tách biệt giao diện người dùng (UI) và xử lý nghiệp vụ (logic/business).
- Giảm sự phụ thuộc giữa các thành phần trong ứng dụng.
- Dễ dàng mở rộng, bảo trì và kiểm thử (testing).

- Tận dụng tối đa các cơ chế của WPF như Data Binding, Command và ObservableCollection.

1.1.2. Cấu trúc và vai trò các thành phần

Model

- Đại diện cho dữ liệu của ứng dụng.
- Không phụ thuộc giao diện người dùng.
- Không chứa logic xử lý UI.
- Thường chỉ gồm các thuộc tính mô tả dữ liệu.

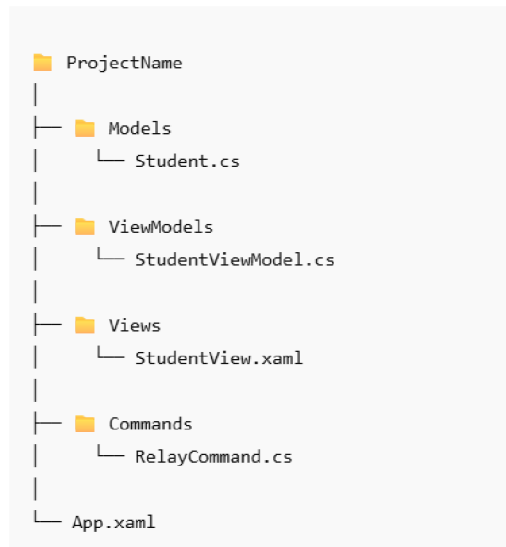
View

- Là giao diện người dùng, được thiết kế bằng XAML.
- Không chứa logic xử lý nghiệp vụ.
- Chỉ thực hiện:
 - Binding dữ liệu từ ViewModel
 - Gọi Command từ ViewModel
 - Định nghĩa Resource, Style, Template (nếu có)

ViewModel

- Là lớp trung gian giữa View và Model.
- Chứa:
 - Các thuộc tính dữ liệu để binding với View
 - Các Command xử lý hành động người dùng
- Không chứa thành phần giao diện (Button, TextBox...).
- Thực hiện giao diện INotifyPropertyChanged để thông báo thay đổi dữ liệu cho View.

Cấu trúc thư mục trong MVVM



1.1.3. Tách lớp Model và ViewModel

- **Model** là lớp mô tả dữ liệu, chứa dữ liệu và logic nghiệp vụ không liên quan tới UI (User Interface).
- **ViewModel** là lớp trung gian giữa UI và Model.

Việc tách rõ Model và ViewModel giúp:

- Dễ bảo trì và mở rộng ứng dụng.
- Dễ kiểm thử (unit test).
- Tránh trộn lẫn logic xử lý và giao diện gây khó hiểu.

1.2. Liên kết dữ liệu trong MVVM

1.2.1. DataBinding

DataBinding là cơ chế liên kết dữ liệu giữa thuộc tính ViewModel và Control trong View (XAML) mà không cần xử lý trực tiếp trong code-behind.

Khi dữ liệu thay đổi ở ViewModel, giao diện sẽ tự động cập nhật và ngược lại.

Các chế độ DataBinding phổ biến

- **OneWay**: Dữ liệu đi từ ViewModel → View, thường dùng cho hiển thị.

Ví dụ: `<TextBlock Text="{Binding Name, Mode=OneWay}" />`

- TwoWay: Dữ liệu 2 chiều giữa View ↔ ViewModel, thường dùng cho nhập liệu.

Ví dụ: `<TextBox Text="{Binding Name, Mode=TwoWay}" />`

🔍 Lưu ý:

- OneTime: Dữ liệu chỉ lấy một lần khi View được khởi tạo..
- OneWayToSource: Dữ liệu chỉ đi từ View → ViewModel.

1.2.2. ObservableCollection<T> và ICollectionView

a. ObservableCollection<T>

- Là một lớp collection động trong .NET, nằm trong namespace:
`using System.Collections.ObjectModel;`
- Dùng thay cho List<T> trong MVVM.
- Khi thêm / xóa / sửa phần tử, giao diện tự cập nhật.
- Thường dùng để binding với: ListBox, ListView, DataGrid

Ví dụ:

```
using System.Collections.ObjectModel;
public class StudentViewModel
{
    public ObservableCollection<Student> Students {get; set;}
    public StudentViewModel()
    {
        Students = new ObservableCollection< Student>();
    }
} //T là kiểu dữ liệu của mỗi phần tử trong collection
```

Trong đó:

`Students.Add("An");` // Thêm phần tử → UI tự cập nhật

`Students.Remove("An");` // Xóa phần tử → UI tự cập nhật

b. ICollectionView:

ICollectionView là lớp trung gian giúp hiển thị dữ liệu từ một collection (thường là `ObservableCollection<T>`) theo các cách khác nhau mà KHÔNG làm thay đổi dữ liệu gốc.

- Dùng để:
 - Lọc (Filter)
 - Sắp xếp (Sort)
 - Phân nhóm (Group)

Ví dụ: Tạo và gán đối tượng `ICollectionView` từ `Students`

```
using System.Windows.Data;  
StudentsView = CollectionViewSource.GetDefaultView(Students);
```

- Khai báo trong `ViewModel`

```
public ICollectionView StudentsView { get; set; }
```

1.2.3. INotifyPropertyChanged

INotifyPropertyChanged cho phép View biết khi nào dữ liệu trong `ViewModel` thay đổi để tự động cập nhật giao diện.

- Là Interface bắt buộc trong `ViewModel`.
- Khi giá trị của một thuộc tính thay đổi, cần gọi phương thức `OnPropertyChanged()`.

Ví dụ:

```
using System.ComponentModel;  
public class StudentViewModel: INotifyPropertyChanged  
{  
    private string name;  
    public string Name  
    {  
        get return name;  
        set {
```

```

        name = value;
        if (PropertyChanged != null)
        {
            PropertyChanged(this, new
                                PropertyChangedEventArgs("Name"));
        }
    }

    //Sự kiện bắt buộc của INotifyPropertyChanged
    public event PropertyChangedEventHandler
                                PropertyChanged;

    // Phương thức gọi sự kiện khi giá trị thuộc tính thay đổi
    protected void OnPropertyChanged(string propertyName)
    {
        if (PropertyChanged != null)
        {
            PropertyChanged(this, new
                                PropertyChangedEventArgs(propertyName));
        }
    }
}

```

🔗 Lưu ý: Khi thuộc tính **Name** thay đổi thì View tự động cập nhật.

2. Bài tập thực hành trên lớp

Bài tập có hướng dẫn

Bài 1. Xây dựng một ứng dụng WPF đơn giản theo mô hình MVVM để hiển thị danh sách sinh viên.

Yêu cầu:

1. Model:

Tạo lớp Student mô tả thông tin sinh viên, gồm các thuộc tính:

- Name (string): Họ và tên sinh viên.
- Age (int): Tuổi sinh viên

2. ViewModel:

Tạo lớp StudentViewModel có nhiệm vụ cung cấp dữ liệu cho View.

ViewModel phải có:

- Thuộc tính: ObservableCollection<Student> Students để chứa danh sách sinh viên.

🔗 Lưu ý: có thể thay bằng List<T>, nhưng chỉ dùng khi dữ liệu KHÔNG thay đổi.

- Khởi tạo sẵn một số sinh viên mẫu trong phương thức khởi tạo.

3. View (XAML):

Thiết kế giao diện hiển thị danh sách sinh viên:



🔗 Lưu ý:

- Sử dụng Datagrid để chứa danh sách sinh viên.
- **Không** viết xử lý trong code-behind.
- **Không** dùng sự kiện Click.
- Chỉ sử dụng Binding.

Hướng dẫn:

Bước 1. Tạo project WPF

Bước 2. Tạo cấu trúc thư mục

Trong project, tạo các folder:

- Model
- ViewModel
- View

Bước 3. Thiết kế giao diện (XAML) theo yêu cầu của đề bài

Trong folder **View**, nhấn chuột phải vào Views → Add → Window (WPF) → đặt tên StudentView.xaml.

Bước 4. Trong folder Model

Tạo class Student.cs (với 02 thuộc tính: Name và Age)

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Bước 5. Trong folder ViewModel

1. Tạo class StudentViewModel.cs

```
using _00_HoVaTen_Tuan5.Model;
public class StudentViewModel
{
    // Danh sách sinh viên để hiển thị lên DataGrid
    public ObservableCollection<Student> Students { get; set; }
    // Constructor
    public StudentViewModel()
    {
        // Khởi tạo dữ liệu mẫu
        Students = new ObservableCollection<Student>
```



```

{
    new Student { Name = "Nguyễn Văn An", Age = 20 },
    new Student { Name = "Trần Ngọc Bình", Age = 20 },
    new Student { Name = "Hoàng Ngọc Chi", Age = 21 }
};
}
}

```

Bước 6. Gán ViewModel cho View (trong xaml)

```

<!-- Gán ViewModel làm DataContext -->
<Window.DataContext>
    <vm:StudentViewModel />
</Window.DataContext>

```

🔍 Lưu ý: khai báo namespace C# trong XAML, giúp XAML nhìn thấy và sử dụng được các class ViewModel

```

xmlns:vm="clr-namespace:_00_HoVaTen_Tuan5.ViewModel"

```

Bước 7. Binding dữ liệu giữa View ↔ ViewModel

- DataGridView sẽ lấy dữ liệu từ danh sách **Students** trong DataContext

```

<DataGridView
    ItemsSource="{Binding Students}"
    ... >

```

- Binding cho từng cột trong Datagrid

Cột Họ và tên:

```

<DataGridTextColumn Header="Họ và tên"
    Binding="{Binding Name}" Width="*" />

```

Cột tuổi:

```

<DataGridTextColumn Header="Tuổi"
    Binding="{Binding Age}" Width="200" />

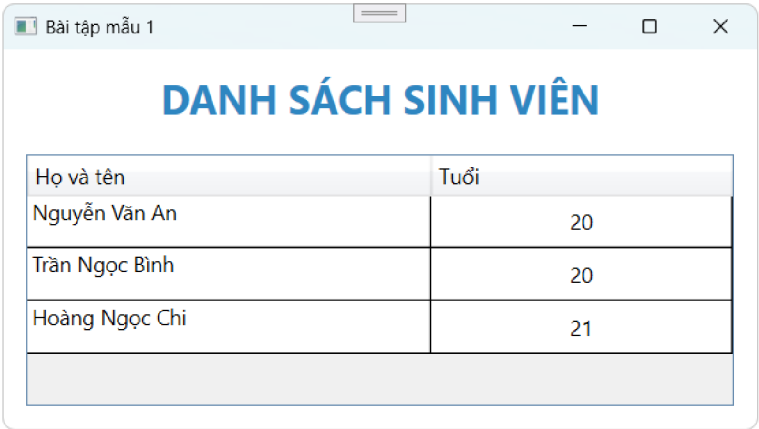
```

Bước 8. Chạy ứng dụng

Trong App.xaml, thay StartupUri thành:

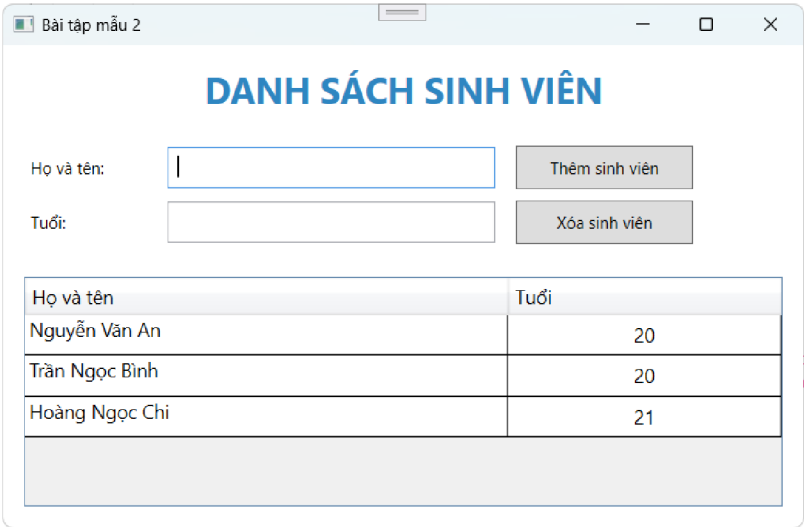
```
StartupUri="View/StudentView.xaml"
```

Kết quả chương trình



Bài 2. Mở rộng ứng dụng ở **Bài 1**, xây dựng chức năng **quản lý sinh viên**, bao gồm:

- Nhập thông tin sinh viên
- Thêm sinh viên mới vào danh sách
- Xóa sinh viên được chọn



Yêu cầu:

1. Ràng buộc dữ liệu (Binding)

- TextBox được binding TwoWay với các thuộc tính tương ứng trong ViewModel
- DataGrid sử dụng:
 - ItemsSource để hiển thị danh sách sinh viên.
 - SelectedItem để xác định sinh viên đang được chọn.
- DataGrid tự động cập nhật khi dữ liệu thay đổi.

2. Xử lý sự kiện

- Sử dụng sự kiện Click cho các Button
- Code-behind chỉ dùng để:
 - Bắt sự kiện Click
 - Gọi các phương thức xử lý trong ViewModel

Hướng dẫn:

Bước 1. Chuẩn bị ViewModel

- Sử dụng lại ViewModel đã xây dựng ở Bài 1
- Bổ sung vào class StudentViewModel các thuộc tính:
 - Thuộc tính lưu danh sách sinh viên
 - Thuộc tính lưu sinh viên đang được chọn
 - Thuộc tính nhận dữ liệu nhập từ TextBox (họ tên, tuổi)

Cách 1: Chỉnh sửa class `StudentViewModel` thực thi Interface `InotifyPropertyChanged` (thông báo cho View khi dữ liệu thay đổi và cập nhật lại UI)

```
public class StudentViewModel : InotifyPropertyChanged
{
    private ObservableCollection<Student> _students;
    public ObservableCollection<Student> Students
    {
        get => _students;
```

```

        set {
            _students = value;
            OnPropertyChanged(nameof(Students));
        }
    }

    // ===== SINH VIÊN ĐƯỢC CHỌN từ DataGridView =====
    private Student _selectedStudent;
    public Student SelectedStudent
    {
        get => _selectedStudent;
        set
        {
            _selectedStudent = value;
            OnPropertyChanged(nameof(SelectedStudent));
        }
    }

    // Nhận dữ liệu từ TextBox, Không lấy TextBox.Text trong code-behind
    private string _newName;
    public string NewName {
        get => _newName;
        set {
            _newName = value;
            OnPropertyChanged(nameof(NewName));
        }
    }

    private int _newAge;
    public int NewAge {
        get => _newAge;
        set {
            _newAge = value;
            OnPropertyChanged(nameof(NewAge));
        }
    }
}

```

```

public StudentViewModel() {
    Students = new ObservableCollection<Student>
    {
        new Student { Name = "Nguyễn Văn An", Age = 20 },
        new Student { Name = "Trần Ngọc Bình", Age = 20 },
    };
}
// ===== INotifyPropertyChanged =====
public event PropertyChangedEventHandler PropertyChanged;
protected void OnPropertyChanged(string propertyName)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
}

```

Cách 2 (khuyến dùng):

- Tạo class `BaseViewModel`

```

public class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
            PropertyChangedEventArgs(propertyName));
    }
}

```

- Chỉnh code cho class `StudentViewModel` kế thừa class `BaseViewModel`

Bước 2. Thiết kế lại giao diện (XAML) theo yêu cầu của đề bài và thực hiện Binding dữ liệu:

- 02 TextBox để nhập: Họ và tên sinh viên, Tuổi

```

<TextBox x:Name="txt_hoten
        Text="{Binding NewName, Mode=TwoWay}" />

```

```
<TextBox x:Name="txt_tuoi"
        Text="{Binding NewAge, Mode=TwoWay}" />
```

- 02 Button: **Thêm** sinh viên, **Xóa** sinh viên đang được chọn

```
<Button x:Name="btn_ThemSV" Content="Thêm sinh viên"
        Click="btn_ThemSV_Click"/>
```

```
<Button x:Name="btn_XoaSV" Content="Xóa sinh viên"
        Click="btn_XoaSV_Click"/>
```

- Databind

```
<DataGrid
        ItemsSource="{Binding Students}"
        SelectedItem="{Binding SelectedStudent, Mode=TwoWay}"
```

Bước 3. Viết code cho chức năng Thêm/ Xóa sinh viên được chọn

- Trong class `StudentViewModel`, bổ sung 02 phương thức `Add()` để thêm sinh viên và `Remove()` để xóa sinh viên được chọn

```
public void AddStudent()
{
    if (string.IsNullOrEmpty(NewName))
        return;
    if (NewAge <= 0)
        return;
    Students.Add(new Student {
        Name = NewName,
        Age = NewAge
    });
}
```

```
// Reset dữ liệu nhập
NewName = string.Empty;
NewAge = 0;
}
```

```
public void RemoveStudent()
{
    if (SelectedStudent == null)
        return;
    Students.Remove(SelectedStudent);
}
```

- Code-behind xử lý các nút (trong **StudentView.xaml.cs**)

1. Khai báo:

```
private StudentViewModel _viewModel = new StudentViewModel();
```

2. Trong phương thức khởi tạo (StudentView())

```
public StudentView ()
{
    InitializeComponent();
    // Lấy ViewModel từ DataContext
    this.DataContext = _viewModel;
    //StudentView hãy lấy dữ liệu mặc định từ object _viewModel
}
```

3. Button Thêm sinh viên

```
private void btn_ThemSV_Click(object sender, RoutedEventArgs e)
{
    _viewModel.AddStudent();
    txt_hoten.Focus();
}
```

4. Button Xóa sinh viên

```
private void btn_XoaSV_Click(object sender, RoutedEventArgs e)
{
    MessageBoxResult r = MessageBox.Show("Bạn muốn xóa sinh viên  
đang chọn", "Xác nhận xóa", MessageBoxButton.YesNo,  
MessageBoxImage.Question);
```

```

if (r == DialogResult.Yes)
    _viewModel?.RemoveStudent();

txt_hoten.Focus();
dtg_dssv.SelectedIndex = 0; //chọn dòng đầu tiên trong datagrid
}

```

Bổ sung chức năng: chọn 1 dòng dữ liệu trong datagrid thì thông tin sinh viên được chọn tự động hiển thị lên TextBox.

Sửa code trong [class StudentViewModel](#)

```

public Student SelectedStudent
{
    get => _selectedStudent;
    set
    {
        _selectedStudent = value;
        if (_selectedStudent != null)
        {
            // Đẩy dữ liệu lên TextBox
            NewName = _selectedStudent.Name;
            NewAge = _selectedStudent.Age;
        }
        OnPropertyChanged(nameof(SelectedStudent));
    }
}

```

Bước 4. Chạy ứng dụng.

Bài 3. Mở rộng bài 2, bổ sung thêm chức năng cho phép người dùng:

- Nhập tên sinh viên cần tìm
- Click nút Lọc để hiển thị các sinh viên phù hợp
- Click nút Sắp xếp (click lần 1 sắp tăng/ click lần 2 sắp giảm theo tuổi)

Bài tập mẫu 3

DANH SÁCH SINH VIÊN

Thông tin sinh viên

Họ và tên:

Nguyễn Văn An

Thêm sinh viên

Tuổi:

20

Xóa sinh viên

Lọc và sắp xếp

Tim tên:

Lọc

Sắp xếp theo tuổi

Họ và tên	Tuổi
Nguyễn Văn An	20
Trần Ngọc Bình	20
Hoàng Ngọc Khánh	21

Hướng dẫn:

Bước 1. Sử dụng Project Bài 2 với Models, ViewModels, View và các class tương ứng.

Bước 2. Bổ sung giao diện (XAML) theo yêu cầu của đề bài.

Bước 3. Trong class `StudentViewModel.cs`, bổ sung

1. `ICollectionView StudentView`: để lọc và sắp xếp

```
private ICollectionView _studentsView;
public ICollectionView StudentsView
{
    get => _studentsView;
    set
    {
        _studentsView = value;
        OnPropertyChanged(nameof(StudentsView));
    }
}
```

17

2. FilterText: nội dung lọc dữ liệu

```
private string _filterText;
public string FilterText
{
    get => _filterText;
    set
    {
        _filterText = value;
        OnPropertyChanged(nameof(FilterText));
    }
}
```

3. Trạng thái sắp xếp

```
private bool _isSortAscending = true;
```

4. Trong phương thức khởi tạo, bổ sung:

```
//Khởi tạo View trung gian từ danh sách sinh viên
StudentsView = CollectionViewSource.GetDefaultView(Students);
```

5. Phương thức lọc sinh viên

```
public void ApplyFilter()
{
    if (StudentsView == null) return;
    StudentsView.Filter = obj =>
    {
        if (string.IsNullOrEmpty(FilterText))
            return true;
        Student sv = obj as Student;
        return sv != null &&
            sv.Name.IndexOf(FilterText,
                StringComparison.OrdinalIgnoreCase) >= 0;
    };
    StudentsView.Refresh();
}
```

6. Phương thức sắp xếp theo tuổi

```
public void SortByAge()
{
    if (StudentsView == null)
        return;
    StudentsView.SortDescriptions.Clear();
    // Nếu đang tăng dần → đổi sang giảm dần, ngược lại
    if (_isSortAscending)
        StudentsView.SortDescriptions.Add(
            new SortDescription("Age", ListSortDirection.Ascending)
        );
    else
        StudentsView.SortDescriptions.Add(
            new SortDescription("Age", ListSortDirection.Descending)
        );
    // Đổi trạng thái cho lần bấm sau
    _isSortAscending = !_isSortAscending;
}
```

Bước 4. Code-behind xử lý nút Lọc và Sắp xếp

1. Button Lọc

```
private void btn_Loc_Click(object sender, RoutedEventArgs e)
{
    _viewModel?.ApplyFilter();
    //nếu _viewModel khác null → gọi ApplyFilter()
    //nếu _viewModel là null → không gọi gì cả, không bị lỗi
}
```

2. Button Sắp xếp

```
private void btn_SapXep_Click(object sender, RoutedEventArgs e)
{
    _viewModel?.SortByAge();
}
```

🔗 Lưu ý: Kiểm tra Binding View ↔ ViewModel

```
<TextBox x:Name="txt_dulieuloc"  
          Text="{Binding FilterText, Mode=TwoWay}" />
```

```
<DataGrid x:Name="dtg_dssv" Grid.Row="3"  
           ItemsSource="{Binding StudentsView}"  
           SelectedItem="{Binding SelectedStudent, Mode=TwoWay}"  
           ... >
```

Bài tập sinh viên tự làm

Bài 4. Xây dựng chương trình Quản lý sinh viên với các chức năng:

- Thêm sinh viên
- Xóa sinh viên (có xác nhận)
- Tìm kiếm sinh viên theo tên
- Thống kê số lượng sinh viên

Yêu cầu:

- **Tải lại trang**
 - Reset toàn bộ control nhập liệu
 - Bỏ chọn DataGrid

- Chuẩn bị trạng thái để thêm sinh viên mới
- **Thêm sinh viên**
 - Phải nhập **đầy đủ thông tin**
 - Sau khi thêm:
 - Chọn dòng sinh viên vừa thêm vào trên datagrid
 - Cập nhật lại tổng số sinh viên
- **Xóa sinh viên**
 - Chỉ xóa khi đã chọn sinh viên trong DataGrid
 - Hiển thị MessageBox xác nhận: *Bạn có chắc chắn muốn xóa sinh viên này không?"*
 - Sau khi xóa:
 - Cập nhật lại tổng số sinh viên
 - Chọn dòng sinh viên đầu tiên trong datagrid
- **Chọn sinh viên**
 - Click chọn 1 dòng trong DataGrid
 - Hiển thị thông tin sinh viên được chọn lên các control nhập liệu
- **Tìm kiếm**
 - Thực hiện **tự động khi nhập dữ liệu vào TextBox** (không sử dụng nút “Lọc”)

Quản lý Sinh viên

QUẢN LÝ SINH VIÊN

Ứng dụng WPF – MVVM cơ bản

Họ và tên:

Nguyễn Bình An

Tuổi:

20

Giới tính:

☒ Nam ☐ Nữ

Thành phố:

Hà Nội

Thêm

Xóa

Tải lại

🔍 Tìm theo tên:

Tổng số sinh viên: 3

Họ tên	Tuổi	Giới tính	Thành phố
Nguyễn Bình An	20	Nam	Hà Nội
Trần Khánh Bình	19	Nữ	Đà Nẵng
Ngô Thị Quỳnh Châu	21	Nữ	TP.HCM

3. Bài tập về nhà

Xây dựng ứng dụng Quản lý công việc cá nhân (Todo List) bằng WPF, cho phép người dùng quản lý danh sách các công việc cần làm.

Mỗi công việc gồm các thông tin:

- Tên công việc
- Mức độ ưu tiên
- Trạng thái hoàn thành
- Ghi chú

The screenshot shows a WPF application window titled "Quản lý công việc" (Task Management). The main content area has a header "QUẢN LÝ CÔNG VIỆC" with the subtitle "Todo List - WPF MVVM". Below the header, there are two main sections: "Nhập công việc" (Add Task) on the left and "Lưu / Tải dữ liệu" (Save / Load Data) on the right. The "Nhập công việc" section contains a text input field, a dropdown menu, a checkbox labeled "Đã hoàn thành" (Completed), another text input field, and three buttons: "Thêm" (Add), "Xóa" (Delete), and "Làm mới" (Refresh). The "Lưu / Tải dữ liệu" section contains two buttons: "Lưu ra JSON" (Save to JSON) and "Tải từ JSON" (Load from JSON). Below these sections is a summary bar with a search icon, a text input field, and a label "Tổng công việc: 2" (Total tasks: 2). At the bottom is a DataGrid with the following data:

Tên công việc	Ưu tiên	Hoàn thành	Ghi chú
Làm bài WPF	Cao	☐	Bài tập tuần này
Đọc tài liệu MVVM	Trung bình	☒	

Yêu cầu chức năng:

1. Thêm công việc mới

- Người dùng nhập thông tin công việc: Tên công việc, Mức độ ưu tiên, Trạng thái hoàn thành, Ghi chú (không bắt buộc).
- Không cho phép thêm nếu chưa nhập tên công việc
- Sau khi thêm thành công:
 - Công việc mới được hiển thị trong danh sách
 - Tự động chọn dòng công việc vừa thêm

2. Xóa công việc

- Chỉ cho phép xóa khi đã chọn công việc trong danh sách
 - Hiện thị hộp thoại xác nhận trước khi xóa
 - Sau khi xóa danh sách công việc được cập nhật.

3. Chọn công việc: Khi người dùng click chọn một dòng trong DataGrid, hiển thị thông tin công việc được chọn lên các control nhập liệu.

Quản lý công việc

QUẢN LÝ CÔNG VIỆC

Todo List – WPF MVVM

Nhập công việc

Độc tài liệu MVVM

Trung bình

☒ Đã hoàn thành

✓ Thêm

✕ Xóa

↺ Làm mới

Lưu / Tải dữ liệu

Lưu ra JSON

Tải từ JSON

Tổng công việc: 2

Tên công việc	Ưu tiên	Hoàn thành	Ghi chú
Làm bài WPF	Cao	☐	Bài tập tuần này
Độc tài liệu MVVM	Trung bình	☒	

4. Tìm kiếm công việc

- Tìm kiếm công việc theo tên công việc
- Thực hiện tự động khi người dùng nhập dữ liệu vào TextBox tìm kiếm
- Khi xóa nội dung trong TextBox tìm kiếm: danh sách hiển thị lại toàn bộ công việc

5. Thống kê

- Hiển thị thông tin thống kê:
 - Tổng số công việc
- Thống kê phải tự động cập nhật khi:
 - Thêm công việc
 - Xóa công việc

6. Làm mới trang

- Xóa toàn bộ dữ liệu trong các ô nhập
- Bỏ chọn công việc đang chọn trong danh sách

7. Lưu và tải dữ liệu (JSON)

Lưu dữ liệu

- Cho phép người dùng lưu danh sách công việc ra file JSON
- Dữ liệu trong file phải đúng cấu trúc, có thể đọc lại được

Tải dữ liệu

- Cho phép người dùng tải dữ liệu từ file JSON
- Sau khi tải:
 - Xóa danh sách hiện tại
 - Nạp lại danh sách từ file
 - Giao diện tự động cập nhật

Hướng dẫn lưu/tải file JSON:

JSON (JavaScript Object Notation) là một định dạng dữ liệu dạng văn bản, dùng để lưu trữ và trao đổi dữ liệu.

1. Cài Newtonsoft.Json bằng NuGet

Right click project → Manage NuGet Packages... → (Browse)
Newtonsoft.Json → Install

2. Sử dụng thư viện: `using Newtonsoft.Json;`

3. Lưu file

```
public void SaveToJson(string filePath)
{
    var json = JsonConvert.SerializeObject(Tasks,
                                           Newtonsoft.Json.Formatting.Indented);
    File.WriteAllText(filePath, json);
}
```

Nút Lưu ra JSON

```
private void btn_LuuJSON_Click(object sender, RoutedEventArgs e)
{
    SaveFileDialog dlg = new SaveFileDialog();
    dlg.Filter = "JSON file (*.json) | *.json";
    dlg.FileName = "todo.json";
    if (dlg.ShowDialog() == true)
    {
        _vm.SaveToJson(dlg.FileName);
        MessageBox.Show("Lưu file thành công!",
            "Thông báo",
            MessageBoxButton.OK,
            MessageBoxImage.Information);
    }
}
```

4. Tải file

```
public void LoadFromJson(string filePath)
{
    if (!File.Exists(filePath))
        return;
    var json = File.ReadAllText(filePath);
    var data = JsonConvert.DeserializeObject<ObservableCollection<TodoItem>>(json);
    Tasks.Clear();
    foreach (var item in data)
        Tasks.Add(item);
}
```

Nút Tải từ JSON

```
private void btn_TaiJSON_Click(object sender, RoutedEventArgs e) {
    OpenFileDialog dlg = new OpenFileDialog();
    dlg.Filter = "JSON file (*.json) | *.json";
    if (dlg.ShowDialog() == true)
        _vm.LoadFromJson(dlg.FileName);
}
```